

FIAP
Engenharia de Software
Computational Thinking with Python

Câmera Inteligente com OCR

Sprint 3 – Documentação Técnica

Integrantes do Grupo:

Artur Fabi Brandi – RM570258

Felipe Santos Ribas – RM569121

Luiz Gonzaga – RM572446

Orion Cavalcante França – RM573677

Victor Lula Heineken Rodrigues – RM570782

São Paulo, 2026

SUMÁRIO

1 Descritivo do Projeto

1.1 Contexto e Motivação

1.2 Objetivo Geral

1.3 Justificativa

1.4 Público-Alvo

2 Descrição das Funcionalidades Implementadas

2.1 Simular Captura de Texto (OCR)

2.2 Pesquisar Texto no Google

2.3 Traduzir Texto Capturado

2.4 Visualizar Histórico de Capturas

2.5 Limpar Histórico

2.6 Ver Estatísticas das Capturas

NOVO

3 Conceitos de Python Aplicados

4 Estrutura do Código-Fonte

5 Justificativa de Alterações em Relação à Sprint 2

NOVO

5.1 O que foi removido

5.2 O que foi modificado

5.3 O que foi adicionado

1. DESCRITIVO DO PROJETO

1.1 Contexto e Motivação

O fluxo de pesquisa acadêmica envolve etapas repetitivas: localizar um texto (em livro, lousa, slide ou documento), digitá-lo manualmente em um buscador ou tradutor, e só

então obter a informação desejada. Esse processo gera fricção desnecessária e dispersa o foco do estudante.

A feature proposta elimina essas etapas ao integrar a câmera do smartphone com ferramentas de pesquisa e tradução, utilizando OCR (Reconhecimento Óptico de Caracteres) para capturar textos do ambiente e convertê-los em ações imediatas.

1.2 Objetivo Geral

Desenvolver, em Python, uma simulação do fluxo de uma câmera inteligente com OCR: captura de texto, armazenamento em histórico, pesquisa no Google e tradução — reduzindo etapas manuais no processo de pesquisa acadêmica. A partir da Sprint 3, o sistema passa também a oferecer uma visão consolidada do uso da câmera por meio de estatísticas das capturas realizadas.

1.3 Justificativa

Os cenários de uso contemplados são diretamente ligados ao cotidiano universitário:

- Capturar conteúdo de lousa e pesquisá-lo sem trocar de aplicativo;
- Digitalizar trechos de livros em língua estrangeira e traduzir sem copiar manualmente;
- Registrar anotações manuscritas de aula para referência posterior;
- Consultar slides projetados em sala de forma rápida;

Enxergar, ao longo do semestre, que tipo de conteúdo o estudante mais captura (impresso, lousa/slide ou manuscrito), a partir da tela de estatísticas. **NOVO — Sprint 3**

O diferencial da solução é transformar a câmera de ferramenta passiva de captura em ferramenta ativa de pesquisa e aprendizado.

1.4 Público-Alvo

Estudantes universitários em aulas presenciais ou híbridas que precisam pesquisar, traduzir ou registrar conteúdos de forma rápida durante o processo de estudo.

2. DESCRIÇÃO DAS FUNCIONALIDADES IMPLEMENTADAS

2.1 Simular Captura de Texto (OCR)

Simula o processo de detecção automática de texto que, em um aplicativo real, seria executado pelo motor OCR ao apontar a câmera para uma superfície. O usuário insere o texto detectado e informa o tipo de fonte.

Regras de negócio:

- Textos em branco ou com menos de 3 caracteres são rejeitados;
- A classificação do tipo de fonte (impresso, lousa/slide, manuscrito) é obrigatória e validada em loop;
- Cada captura é armazenada como dicionário na lista `historico`, recebida por parâmetro.

2.2 Pesquisar Texto Capturado no Google

Gera a URL de pesquisa no Google para um texto previamente capturado, simulando a ação contextual que apareceria na interface da câmera após detecção.

Regras de negócio:

- Disponível apenas se houver ao menos uma captura no histórico;
- O usuário seleciona qual captura pesquisar; a entrada é validada em loop;
- O texto é codificado via `urllib.parse.quote` para compor a URL de forma segura.

2.3 Traduzir Texto Capturado

Gera um link direto para o Google Translate com o texto e o idioma de destino já configurados, eliminando a necessidade de copiar e colar manualmente.

Regras de negócio:

- Disponível apenas se houver capturas no histórico;
- Idiomas suportados: Inglês, Espanhol, Francês e Alemão;
- Seleção de texto e de idioma são validadas em loop até entrada correta;
- A URL é gerada com o código ISO 639-1 do idioma via f-string.

2.4 Visualizar Histórico de Capturas

Exibe todas as capturas registradas na sessão, com índice, tipo de fonte e texto completo, permitindo ao usuário revisar o conteúdo detectado anteriormente.

Regras de negócio:

- Se o histórico estiver vazio, uma mensagem informativa é exibida;

- A listagem percorre a lista com `for/ enumerate`, formatando cada item com f-string;
- O total de registros é exibido antes da listagem.

2.5 Limpar Histórico

Remove todos os registros da lista de capturas após confirmação explícita, prevenindo apagamentos acidentais.

Regras de negócio:

- Se o histórico estiver vazio, a operação é informada sem solicitar confirmação;
- O total de itens a remover é exibido na pergunta de confirmação;
- Apenas a resposta "s" confirma; qualquer outra entrada cancela a operação.

2.6 Ver Estatísticas das Capturas NOVO — Sprint 3

Agrupar as capturas registradas por tipo de fonte (impresso, lousa/slide, manuscrito), utilizando um dicionário de contagem construído a partir da lista de capturas do histórico.

Regras de negócio:

- Se o histórico estiver vazio, uma mensagem informativa é exibida e nenhuma estatística é calculada;
- A contagem por tipo é feita pela função `contar_capturas_por_tipo(historico)`, que retorna um dicionário `{tipo: quantidade}`;
- Para cada tipo encontrado, são exibidos a quantidade absoluta de capturas e o percentual em relação ao total da sessão.

3. CONCEITOS DE PYTHON APLICADOS

Conceito	Aplicação no Projeto
Manipulação de variáveis	Texto capturado, tipo de fonte, índice de seleção, URL gerada
Entrada/saída com f-string	Todas as mensagens de feedback e URLs exibidas ao usuário

Conceito	Aplicação no Projeto
Validação de entrada	Funções <code>texto_valido()</code> e <code>indice_valido()</code> isolam as regras de validação e são reutilizadas em múltiplos fluxos do programa
Estruturas de decisão	<code>match-case</code> no menu principal; <code>if-elif-else</code> nas validações internas
Estruturas de repetição	<code>while True</code> no menu e nas validações; <code>for/enumerate</code> na listagem
Listas e Dicionários ampliado	<code>historico</code> : lista de dicionários com <code>append</code> , <code>clear</code> e iteração; dicionários fixos <code>TIPOS_FONTE</code> e <code>IDIOMAS</code> ; e o novo dicionário de contagem retornado por <code>contar_capturas_por_tipo()</code>
Parâmetros e retorno de funções NOVO	<code>historico</code> deixou de ser variável global e passou a ser recebido por parâmetro em todas as funções; <code>criar_captura()</code> , <code>montar_url_busca_google()</code> , <code>montar_url_traducao()</code> e <code>contar_capturas_por_tipo()</code> retornam valores em vez de apenas imprimir
Organização do código	Funções por responsabilidade única; <code>selecionar_captura()</code> reutilizada em duas funcionalidades; <code>ler_tipo_fonte()</code> e <code>ler_idioma_destino()</code> extraídas para eliminar validações duplicadas
Comentários e usabilidade	Docstrings em todas as funções; prefixos <code>[!]</code> para erros e <code>[✓]</code> para sucesso; type hints adicionados em todas as assinaturas

4. ESTRUTURA DO CÓDIGO-FONTE

O arquivo `projeto.py` é dividido em funções com responsabilidade única. Desde a Sprint 3, o histórico de capturas deixou de ser uma variável global e passou a ser criado localmente em `main()`, sendo repassado como parâmetro para todas as demais funções — o que também tornou explícito o uso de valores de retorno em boa parte das funções auxiliares.

Função	Responsabilidade
<code>exibir_menu()</code>	Renderiza o menu principal com todas as opções numeradas
<code>texto_valido(texto)</code> NOVO	Retorna <code>True</code> se o texto tiver ao menos 3 caracteres úteis
<code>resumir_texto(texto, limite=45)</code> NOVO	Retorna o texto truncado para exibição em listas, com valor padrão para o limite
<code>criar_captura(texto, tipo)</code> NOVO	Monta e retorna o dicionário que representa uma captura
<code>ler_tipo_fonte()</code> NOVO	Solicita e valida em loop o tipo de fonte; retorna o nome selecionado
<code>simular_captura_ocr(historico)</code>	Coleta e valida texto e tipo de fonte; adiciona ao <code>historico</code> recebido por parâmetro e retorna a captura criada (ou <code>None</code>)
<code>indice_valido(escolha, tamanho)</code> NOVO	Retorna <code>True</code> se a escolha for um índice válido dentro do histórico
<code>selecionar_captura(historico, acao)</code>	Auxiliar: lista capturas disponíveis e retorna a escolhida, com validação
<code>montar_url_busca_google(texto)</code> NOVO	Retorna a URL de pesquisa no Google para o texto informado
<code>pesquisar_no_google(historico)</code>	Usa <code>selecionar_captura()</code> e <code>montar_url_busca_google()</code> para exibir o link de busca
<code>ler_idioma_destino()</code> NOVO	Solicita e valida em loop o idioma de destino; retorna nome e código ISO
<code>montar_url_traducao(texto, idioma_codigo)</code> NOVO	Retorna a URL do Google Translate para o texto e idioma informados
<code>traduzir_texto(historico)</code>	Usa <code>selecionar_captura()</code> , <code>ler_idioma_destino()</code> e <code>montar_url_traducao()</code> para exibir o link de tradução

Função	Responsabilidade
<code>visualizar_historico(historico)</code>	Itera sobre o <code>historico</code> recebido por parâmetro e exibe todas as capturas formatadas
<code>contar_capturas_por_tipo(historico)</code> NOVO	Retorna um dicionário <code>{tipo: quantidade}</code> a partir do histórico
<code>visualizar_estatisticas(historico)</code> NOVO	Exibe quantidade e percentual de capturas por tipo de fonte
<code>limpar_historico(historico)</code>	Solicita confirmação e executa <code>historico.clear()</code>
<code>main()</code>	Cria o histórico local, exibe o loop principal com <code>match-case</code> ; ponto de entrada do programa

Loop principal

```
def main():
    historico_capturas = []
    print("Bem-vindo ao Sistema de Câmera Inteligente com OCR!")

    while True:
        exibir_menu()
        opcao = input("Escolha uma opção: ").strip()

        match opcao:
            case "1": simular_captura_ocr(historico_capturas)
            case "2": pesquisar_no_google(historico_capturas)
            case "3": traduzir_texto(historico_capturas)
            case "4": visualizar_historico(historico_capturas)
            case "5": visualizar_estatisticas(historico_capturas)  #
NOVO
            case "6": limpar_historico(historico_capturas)
            case "0":
                print("Encerrando o sistema. Até logo!")
                break
            case _:
                print("[!] Opção inválida.")
```

Fluxo de execução

1. O programa inicia por `main()`, chamada via `if __name__ == "__main__"`.
2. Em `main()`, o histórico de capturas é criado como lista local (`historico_capturas`) — na Sprint 2 essa lista era uma variável global do módulo.
3. O menu é exibido em loop até que o usuário selecione a opção **0 (Sair)**.
4. Cada opção executa a função correspondente, sempre recebendo `historico_capturas` como parâmetro, e retorna ao loop principal.
5. O histórico persiste durante toda a sessão, mas seu escopo passou a ser local a `main()`, sendo repassado explicitamente a cada função que precisa lê-lo ou modificá-lo.

5. JUSTIFICATIVA DE ALTERAÇÕES EM RELAÇÃO À SPRINT 2

NOVO

Esta seção documenta, conforme exigido pelo Sprint 3 e 4 Challenge, o que foi removido, modificado e adicionado no código em relação à versão entregue na Sprint 2.

5.1 O que foi removido

- A variável global `historico_capturas`, substituída por um parâmetro `historico` recebido explicitamente por cada função a partir de `main()`.
- Os blocos de validação de tipo de fonte e de idioma que estavam duplicados dentro de `simular_captura_ocr()` e `traduzir_texto()`; ambos foram extraídos para funções próprias.

5.2 O que foi modificado

- Todas as funções que manipulam o histórico passaram a recebê-lo como parâmetro e, quando aplicável, a retornar valores: `simular_captura_ocr()` agora retorna o dicionário criado (ou `None` se o texto for inválido); `criar_captura()`, `montar_url_busca_google()` e `montar_url_traducao()` retornam os dados/URLs montados em vez de apenas imprimi-los.

- A validação do texto capturado e do índice escolhido pelo usuário foi isolada nas funções puras `texto_valido()` e `indice_valido()`, reutilizadas em múltiplos pontos do programa.
- O menu principal foi renumerado para acomodar a nova opção de estatísticas: "Limpar histórico" passou da opção 5 (Sprint 2) para a opção 6 (Sprint 3).
- Foram adicionados type hints (`-> dict`, `-> bool`, `-> str`, `-> dict | None`) e docstrings revisadas em todas as funções, como boa prática de programação.

5.3 O que foi adicionado

- Nova funcionalidade **"Ver estatísticas das capturas"** (opção 5 do menu, ver seção 2.6), que utiliza a função `contar_capturas_por_tipo()` para agrupar as capturas em um dicionário `{tipo: quantidade}` e exibir quantidade e percentual por tipo de fonte.
- Uso mais explícito de dicionários como estrutura de dados central, além da lista de capturas: os dicionários `TIPOS_FONTE` e `IDIOMAS` passaram a ser percorridos via `.items()` dentro de funções dedicadas (`ler_tipo_fonte()` e `ler_idioma_destino()`), e foi criado o novo dicionário de contagem retornado por `contar_capturas_por_tipo()`.
- Função auxiliar `resumir_texto()`, extraída do corpo de `selecionar_captura()`, com parâmetro de valor padrão (`limite: int = 45`) e retorno de valor, deixando explícito o uso de parâmetros e retorno de funções.